

L'arbre de décision dans XGBoost : construction, rôle et facteurs de performance

Document de travail — réponses structurées pour le mémoire sur le gradient boosting, appuyées sur les ressources du projet (Chen & Guestrin, 2016 ; Friedman, 2001 ; notebook d'implémentation)

1 Comment l'arbre est-il construit dans XGBoost ?

Dans XGBoost, chaque arbre est un arbre de régression (CART) construit de façon gloutonne, nœud par nœud, en optimisant à chaque étape une fonction objectif régularisée. Le modèle complet prédit la somme des scores de K arbres, $\hat{y}_i = \sum_k f_k(x_i)$, et l'objectif pénalise à la fois l'erreur de prédiction et la complexité de chaque arbre :

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

où T est le nombre de feuilles de l'arbre et w le vecteur des scores de ses feuilles. Comme les arbres sont des fonctions et non des paramètres numériques, ils ne peuvent pas être appris par une descente de gradient classique : l'apprentissage est additif, un arbre f_t est ajouté à chaque itération t pour corriger la prédiction courante. La contribution centrale de XGBoost par rapport au gradient boosting original de Friedman est d'utiliser une approximation de Taylor du **second ordre** de la perte, ce qui fait apparaître pour chaque exemple i un gradient g_i et une dérivée seconde (hessienne) h_i . Pour une structure d'arbre donnée, ces statistiques permettent de calculer analytiquement le score optimal de chaque feuille j :

$$w^*_j = -G_j / (H_j + \lambda), \text{ avec } G_j = \sum_{i \in j} g_i \text{ et } H_j = \sum_{i \in j} h_i$$

Cette formule joue le même rôle que l'impureté de Gini dans un arbre de classification classique, mais se généralise à n'importe quelle fonction de perte différentiable.

Il est impossible d'énumérer toutes les structures d'arbre possibles : XGBoost utilise donc un algorithme glouton qui démarre d'un nœud unique et ajoute itérativement le split qui réduit le plus la perte. Pour un split séparant les exemples en sous-ensembles gauche et droit, le gain apporté est :

$$L_{split} = \frac{1}{2} [G_l^2 / (H_l + \lambda) + G_r^2 / (H_r + \lambda) - G^2 / (H + \lambda)] - \gamma$$

Cette formule est évaluée pour chaque variable et chaque seuil candidat ; le split retenu est celui qui maximise le gain. Le terme γ agit comme un coût fixe par feuille ajoutée : si aucun split ne produit un gain positif, le nœud reste une feuille. Deux algorithmes calculent ces candidats : l'algorithme glouton exact énumère tous les seuils possibles après avoir trié les données (coûteux mais optimal), tandis que l'algorithme approché propose un nombre réduit de seuils à partir de quantiles pondérés par h_i de la distribution de chaque variable, ce qui permet de traiter des volumes de données qui ne tiennent pas en mémoire. Enfin, XGBoost gère les valeurs manquantes par un algorithme conscient de la sparsité : pour chaque nœud, une direction par défaut est apprise directement à partir des données plutôt que fixée arbitrairement, ce qui évite toute imputation préalable.

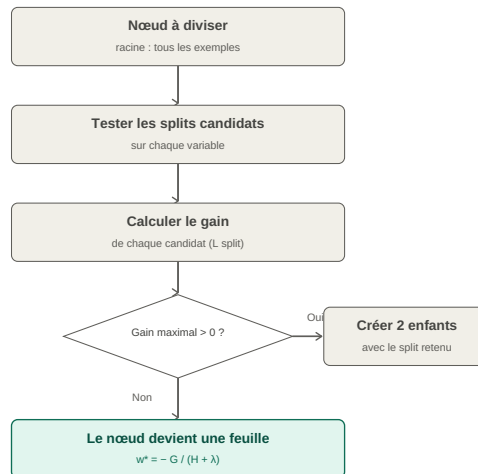


Figure 1. Boucle de construction gloutonne d'un nœud de l'arbre. Ce processus est répété récursivement sur chaque nouveau nœud enfant, jusqu'à une condition d'arrêt : profondeur maximale atteinte, gain ≤ 0 , ou nombre d'exemples insuffisant dans une feuille (voir section 5).

2 Pourquoi utiliser un arbre de décision comme apprenant faible ?

Le cadre du gradient boosting proposé par Friedman (2001) est en principe indépendant du choix de l'apprenant faible $h(x)$, mais l'arbre de régression s'est imposé comme le choix quasi universel, pour des raisons qui se recoupent depuis l'article fondateur jusqu'à XGBoost.

Premièrement, un arbre est invariant à toute transformation monotone d'une variable d'entrée : utiliser x , $\log(x)$ ou e^x comme variable produit exactement les mêmes splits, puisque seul l'ordre des valeurs compte. Cela supprime le besoin de deviner et de tester des transformations de variables avant l'entraînement. Deuxièmement, et c'est le point le plus important pour répondre à la question 4 ci-dessous, un arbre découpe l'espace des variables en régions disjointes et prédit une constante dans chaque région, ce qui lui permet de représenter des effets non linéaires et des interactions entre variables sans que ces interactions aient besoin d'être spécifiées à l'avance. Troisièmement, les arbres gèrent nativement les valeurs manquantes et restent robustes face à des variables d'entrée non pertinentes, qui ne sont simplement jamais sélectionnées pour un split.

Ce dernier point a une contrepartie importante, illustrée par les expérimentations du notebook du mémoire : sur une seule variable (la qualité de construction, OverallQual), un arbre à un seul split (decision stump) atteint une MSE de test de 0,094533, ce qui est légèrement *moins bon* qu'une simple régression linéaire sur cette même variable unique (0,085807). Avec une seule variable, il n'existe par définition aucune interaction à exploiter, et l'approximation « en escaliers » d'un arbre n'apporte alors aucun avantage face à une droite continue. L'avantage des arbres n'apparaît pas dans un split isolé, mais dans l'accumulation de nombreux splits peu profonds par boosting, ainsi que dans l'usage de plusieurs variables simultanément — ce que développe la section 4.

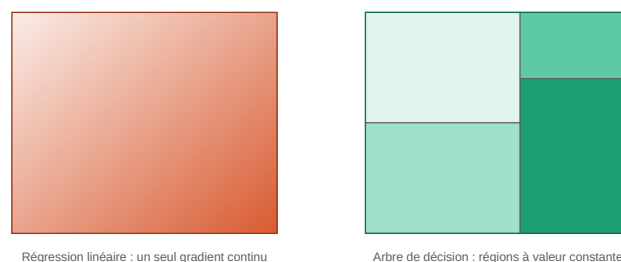


Figure 2. Représentation schématique de l'espace des variables. Une régression linéaire impose un unique gradient continu sur tout l'espace ; un arbre découpe cet espace en régions à valeur constante, dont les frontières et les niveaux sont appris à partir des données.

3 Comment l'arbre réagit-il ?

On peut distinguer deux moments où l'arbre « réagit » : pendant son apprentissage, où il réagit au signal d'erreur du modèle courant, et pendant la prédiction, où il réagit à un nouvel exemple.

Réaction au signal d'erreur, pendant l'apprentissage

À chaque itération t , l'arbre ne voit jamais directement la cible y : il reçoit pour chaque exemple i un gradient g_i et une hessienne h_i , calculés à partir de l'écart entre la prédiction courante $\hat{y}_i^{(t-1)}$ et la cible. Ces statistiques mesurent à quel point le modèle se trompe sur chaque exemple, et avec quelle confiance. La procédure de construction réagit directement à ce signal : un split n'est intéressant, au sens de la formule L_{split} , que s'il sépare des exemples aux statistiques de gradient différentes — c'est-à-dire s'il isole un sous-groupe que le modèle sur- ou sous-estime systématiquement. Là où le modèle est déjà précis (gradients proches de zéro), aucun split ne produit de gain utile ; là où l'erreur résiduelle est grande et structurée selon une variable, un split à cet endroit produit un gain élevé. C'est ce mécanisme qui fait que chaque nouvel arbre se concentre, sans qu'on ait besoin de le lui dire explicitement, sur les cas que l'ensemble des arbres précédents traite encore mal.

Réaction à un nouvel exemple, pendant la prédiction

Une fois l'arbre entraîné, sa structure est fixée. Prédire pour un nouvel exemple consiste simplement à évaluer, à chaque nœud en partant de la racine, la condition apprise sur une variable ($x_j < \text{seuil} ?$), et à suivre la branche correspondante jusqu'à une feuille. La « réaction » de l'arbre à cet exemple est alors le score w_j précalculé de cette feuille, une constante partagée par tous les exemples qui y aboutissent. Deux maisons ne différant que sur la variable testée au premier split peuvent ainsi finir dans des feuilles complètement différentes, et recevoir des corrections très différentes — c'est ce qu'illustre la figure 3.

Pour une valeur manquante, l'arbre ne réagit pas non plus par un échec ou par une valeur imputée arbitrairement : il suit la direction par défaut apprise pour ce nœud pendant l'entraînement, c'est-à-dire la direction qui, empiriquement, minimisait la perte sur les exemples ayant cette valeur manquante. Le comportement de l'arbre face à une donnée incomplète est donc lui aussi appris, et non codé en dur.

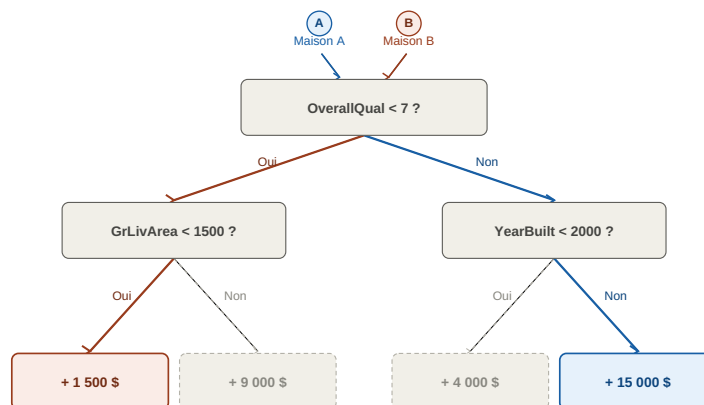


Figure 3. Le même arbre entraîné réagit différemment selon l'exemple présenté : deux maisons aux caractéristiques différentes empruntent des chemins différents et reçoivent des corrections différentes. □ chemin de la maison A □ chemin de la maison B branches non empruntées

4 Quel problème résout-on, et en quoi une régression de base est-elle insuffisante ?

Le problème général de l'apprentissage supervisé est d'estimer une fonction $F(x)$ qui minimise l'espérance d'une perte entre $F(x)$ et y , sur la distribution jointe (inconnue) de (x, y) . Friedman formule précisément le gradient boosting comme une optimisation numérique menée directement dans l'**espace des fonctions** plutôt que dans l'espace des paramètres : à chaque itération, le gradient négatif de la perte par rapport aux valeurs courantes de la fonction (le pseudo-résidu) joue le rôle que joue le gradient négatif d'une perte paramétrique dans une descente de gradient ordinaire — sauf qu'ici, on entraîne une nouvelle fonction (un arbre) pour approximer ce gradient, plutôt que de simplement avancer d'un pas le long d'une direction fixe.

Une régression linéaire fixe à l'avance la forme de cette fonction : $F(x) = \beta_0 + \sum_j \beta_j x_j$. Chaque terme est additif, et l'effet de chaque variable x_j est constant pour toute valeur des autres variables (le même β_j partout dans l'espace). C'est une hypothèse structurelle forte sur la fonction réelle $F^*(x) = E[y|x]$, rarement vérifiée par les phénomènes réels. Si la relation vraie comporte un effet non linéaire (l'effet marginal de la surface diminue pour les très grandes maisons) ou un effet d'interaction (une finition de qualité élevée vaut davantage sur une grande maison que sur une petite), un modèle linéaire ne peut représenter ni l'un ni l'autre sans que ces termes aient été construits à l'avance par le modélisateur — et un mauvais choix de transformation introduit un biais qu'aucune quantité de données supplémentaires ne corrige.

Un arbre de régression suffisamment grand peut, à l'inverse, approximer localement n'importe quelle fonction sans hypothèse de forme préalable : c'est un approximateur non paramétrique par morceaux constants. Le gradient boosting renforce encore cela en accumulant de

nombreux petits arbres plutôt que d'en construire un seul : chaque nouvel arbre n'a qu'à expliquer la part du signal que la somme courante ne capture pas encore, ce qui correspond, selon Friedman, à répéter de petits pas de descente la plus raide dans l'espace des fonctions.

Le tableau suivant reprend les résultats obtenus dans le notebook du mémoire sur le jeu de données Kaggle House Prices (MSE sur le jeu de test, en log-prix), et illustre concrètement ce qui précède.

Modèle	MSE (test)
Gradient Boosting sklearn (200 arbres, profondeur 3, lr = 0,05)	0,022168
Régression linéaire (9 variables)	0,023707
Gradient Boosting from scratch (568 stumps, optimal)	0,023609
Random Forest (100 arbres)	0,024225
Gradient Boosting from scratch (100 stumps, lr = 0,1)	0,029390
Gradient Boosting sklearn (100 stumps, profondeur 1, lr = 0,1)	0,029426
Arbre de décision seul (profondeur 4)	0,045743
Régression linéaire (1 variable : OverallQual)	0,085807
Decision Stump isolé (1 split, 1 variable)	0,094533

Trois lectures de ce tableau confirment le raisonnement théorique. D'abord, le stump isolé est bien le pire modèle : sans interaction à exploiter et sans accumulation par boosting, l'arbre n'apporte rien. Ensuite, un arbre unique mais plus profond (profondeur 4) progresse beaucoup, mais reste loin derrière la régression linéaire utilisant toutes les variables — un arbre seul demeure une approximation par morceaux relativement grossière et instable, ce que Friedman identifie explicitement comme le principal défaut des modèles à arbre unique. Enfin, l'accumulation de nombreux arbres peu profonds par boosting finit par dépasser la régression linéaire complète (dès 568 stumps), et un boosting utilisant des arbres légèrement plus expressifs (profondeur 3) l'emporte nettement sur tous les autres modèles testés — confirmant que c'est la combinaison « petits arbres + boosting », et non l'arbre seul, qui résout le problème que la régression de base ne peut pas résoudre.

5 Qu'est-ce qui fait qu'un arbre fonctionne mieux ? Profondeur, largeur, et autres leviers

Plusieurs leviers, distincts mais complémentaires, déterminent la performance d'un arbre dans un modèle de boosting. Aucun n'agit isolément : ils doivent être pensés ensemble.

La taille de chaque arbre (profondeur, nombre de feuilles J)

Friedman montre que le nombre de feuilles terminales J d'un arbre contrôle directement l'ordre maximal d'interaction entre variables que ce modèle peut capturer : un arbre à J feuilles peut représenter des interactions d'ordre au plus $J - 1$. Comme la plupart des fonctions rencontrées en pratique sont dominées par des interactions d'ordre faible, des arbres relativement petits (J entre 4 et 8, typiquement) suffisent généralement et donnent de meilleurs résultats que des arbres plus grands, qui ajoutent du bruit sans gain de précision réel. Dans XGBoost, le paramètre correspondant est la profondeur maximale (`max_depth`, généralement entre 3 et 10, avec une valeur par défaut de 6) : plus elle est grande, plus chaque arbre peut représenter des interactions complexes, mais plus il risque aussi d'apprendre le bruit du résidu courant plutôt que le signal utile.

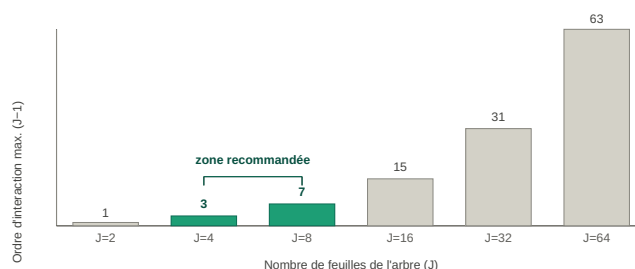


Figure 4. Ordre d'interaction maximal représentable ($J-1$) en fonction du nombre de feuilles J de l'arbre (Friedman, 2001). La zone $J = 4$ à 8 est généralement suffisante pour les fonctions rencontrées en pratique.

Le nombre d'arbres (M) et le taux d'apprentissage (η)

Le shrinkage introduit par Friedman réduit la contribution de chaque nouvel arbre d'un facteur η avant de l'ajouter au modèle. Un η plus petit donne en général une meilleure performance finale, au prix d'un nombre d'itérations M plus grand pour l'atteindre — le compromis entre les deux doit être ajusté conjointement, typiquement via un jeu de validation. Les expérimentations du notebook illustrent bien cette dynamique sur 1 000 itérations à $\eta = 0,1$: l'erreur de test diminue fortement entre 10 arbres (0,102) et 100 arbres (0,029), continue de s'améliorer jusqu'à un minimum atteint à 568 arbres (0,0236), puis remonte très légèrement à 1 000 arbres (0,0238, soit +0,7 %) — la signature classique du surapprentissage une fois que les arbres supplémentaires commencent à modéliser le bruit du résidu plutôt que son signal restant.

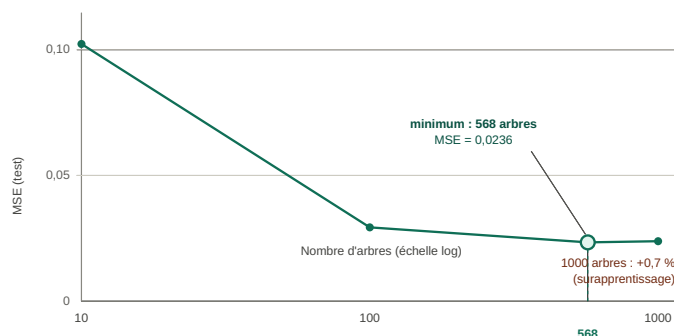


Figure 5. Évolution de la MSE de test en fonction du nombre d'arbres (échelle log), GB from scratch, profondeur 1, $\eta = 0,1$.
Données issues du notebook du mémoire.

Les autres leviers de régularisation propres à XGBoost

Au-delà de la taille de l'arbre et du couple (M, η) , XGBoost expose plusieurs paramètres qui agissent directement dans la formule de gain ou sur la croissance de l'arbre :

γ (gamma)	Gain minimal requis pour accepter un split — agit comme un coût de complexité par feuille ajoutée, directement intégré dans L_{split} ; un nœud reste une feuille si aucun split ne dépasse ce seuil.
λ (lambda)	Pénalité L2 sur les poids des feuilles w — lisse et réduit l'amplitude des scores, en particulier pour les feuilles à faible H_j (peu d'exemples ou exemples peu informatifs).
min_child_weight	Somme minimale de h_i requise dans un nœud enfant — empêche la création de feuilles trop petites, soutenues par trop peu d'exemples pour être fiables.
subsample / colsample	Sous-échantillonnage des lignes ou des colonnes avant chaque arbre, technique reprise des forêts aléatoires — décorrèle les arbres successifs ; le sous-échantillonnage de colonnes s'est révélé, empiriquement, encore plus efficace contre le surapprentissage que celui des lignes.

Ces leviers ne sont pas redondants avec la profondeur ou le nombre d'arbres : ils contrôlent des aspects différents de la complexité du modèle (la confiance accordée à chaque feuille, le coût d'ajouter une feuille, la corrélation entre arbres successifs), et c'est leur combinaison, plus que le réglage isolé d'un seul d'entre eux, qui détermine si l'ensemble final généralise bien au-delà du jeu d'entraînement.

À retenir : un arbre de boosting fonctionne mieux non pas en étant individuellement plus précis, mais en restant volontairement faible et peu profond ($J \approx 4-8$, profondeur $\approx 3-8$), pour que la performance vienne de l'accumulation de nombreuses corrections petites et régularisées (η faible, M grand, $\gamma/\lambda/\text{min_child_weight}$ bien réglés) plutôt que d'un seul arbre cherchant à tout expliquer à lui seul.

Sources

Chen, T., & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16).

Friedman, J. H. (2001). *Greedy Function Approximation: A Gradient Boosting Machine*. Annals of Statistics, 29(5), 1189–1232.

De Cock, D. (2011). *Ames Housing dataset*. Utilisé via la compétition Kaggle « House Prices: Advanced Regression Techniques ».

Résultats expérimentaux : notebook d'implémentation du mémoire (régression linéaire, decision stump, Gradient Boosting from scratch et sklearn, Random Forest).

Outils utilisés : ce document de travail a été préparé avec l'assistance de Claude (Anthropic), à partir des ressources du projet (articles originaux, résumés, notebook d'implémentation). Il est destiné à structurer la compréhension et la préparation de la soutenance orale ; son contenu théorique a été vérifié par recoupement avec les sources citées ci-dessus et doit être reformulé avec vos propres mots avant toute intégration dans le corps du mémoire final.